CMPE243-S02 LOGIC CIRCUIT DESIGN

PROJECT REPORT

Final Project

Finite State Machine-Based Factory Production Control

Submitted by:

Zeynep Nisa Kaya (CMPE)

Şimal Ersuz (CMPE)

Zeynep Bayraktar (CMPE)

Project Supervisor:

Dr. Fatih Genç

Project Mentors:

Assoc.Prof. Dr. Taner Arsan

Prof. Dr. Nima Jafari Navimipour

Assist. Prof. Dr. İlktan Ar

Faculty of Engineering and Natural Sciences

Kadir Has University

Fall 2025

TABLE OF CONTENTS

1 INTRODUCTION □ 5

1.1 Aim of the Project. □ 5

1.2 Overview of Finite State Machines □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ 5

1.3 Mearly FSM Concept □ .. 5

1.4 Description of the Production Process □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ 6

2 PROBLEM DEFINITION □ ... 7

2.1 Machine Order for Product-2 □ .. 7

2.2 Input Signals □ 7

2.3 Output Signals □ . 8

2.4 Design Constraints and Assumptions □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ .. 9

3 STATE DEFINIFTIONS AND STATE DIAGRAM □ □ □ □ □ □ □ □ □ □ □ □ 9

3.1 Definition of System States □ . 9

3.2 Description of Each State □ 10

3.3 State Transition Explanation □ ... 10

3.4 State Diagram (Mearly FSM) □ 12

4 STATE ENCODING □ ...□ □ □ □ □ □ . 12

4.1 Selection of Number of Flip-Flops □ .. 12

4.2 Binary Encoding States □ □ □ □ □ ....□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ 13

4.3 Unused States □ 14

5 NEXT STATE (TRANSITION) TABLE □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ . 14

5.1 Construction of the Transition Table. □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ ... 14

5.2 Input Conditions for State Transitions □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ 16

5.3 Final Next-State Table □ ...□ □ □ □ .. 16

6 FLIP-FLOP SELECTION AND EXCITATION TABLE □ □ □ □ □ □ □ □ □ □ □ 16

6.1 Selection of D Flip-Flop □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ ...□ □ □ □ □ □ ...□ □ □ □ 16

6.2. Relation Between Next State and D Inputs □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ . 18

6.3 Final Excitation Table □ 18

7 MINIMIZATION OF NEXT-STATE LOGIC □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ 19

7.1 Karnaugh Map Method 18
7.2 Minimization of D_2 Logic 19
7.3 Minimization of D_1 Logic 20
7.4 Minimization of D_0 Logic 21
7.5 Summary of Minimized Next-State Equations 22
8 OUTPUT TABLE AND OUTPUT LOGIC 24
8.1 Definition of Output Signals 24
8.2 Mearly Output Behavior 24
8.3 Output Table. 24
8.4 Minimization of Output Logic 25
9 LOGIC CIRCUIT DESIGN 27
9.1 Overall System Block Diagram 27
9.2 Next-State Logic Circuit 27
9.2.1 Logic Diagram of D2 27
9.2.2 Logic Diagram of D1 29
9.2.3 Logic Diagram of D0 31
9.3. Output Logic Circuit 34
9.4. D Flip-Flop Register and Initialization 36
9.5 Complete Logic Diagram 36
10 SIMULATION AND VERIFICATION 38
10.1 MATLAB/Simulink Model Description 38
10.1.1 START Signals (output of the PLC) 38
10.1.2 DONE Signals (input of the PLC) 39
10.2 Simulation Setup 40
10.2.1 Solver Discrete-Time Operation 40
10.2.2 Clock Signal (clk_in) 40
10.2.3 Use of Unit Delay (1/z) 41
10.2.4 Use of Flip-Flops in Machine Subsystems (DFF Blocks) 43
10.2.5 Parameter N (Machine Duration) 43
10.3 Input Test Scenarios 44
10.4 Simulation Results 44

10.4.1 Stateflow Active State Observation 44
10.4.2 START Signals Behavior 45
10.4.3 DONE Signals Behavior 45
10.4.4 Machine Internal Timing Behavior 46
10.5 Verification of Correct Operation 46
11 DISCUSSION 49
11.1 Design Observations 49
11.2 Advantages of the Proposed FSM 49
11.3 Possible Improvements 50
12 CONCLUSION 51
12.1 Summary of the Design 51
12.2 Final Remarks 51
REFERENCES 53

1 INTRODUCTION

1.1 Aim of the Project

The aim of this section is to design a production control system for Product-2 using a finite state machine (FSM) in which the order of operation of the multiple machines is controlled so that one machine's operation has to be completed before another machine is allowed to operate. The controller is implemented with D flip-flops based on a finite state machine (FSM).

1.2 Overview of Finite State Machines

An FSM is a mathematical computation model that can be defined as an abstract machine consisting of a finite number of states, such that the states may change in response to input conditions or clock signals. Because of their simplicity and regularity, FSMs are a popular way to design digital systems.

An FSM consist of:

- Inputs
- Finite number of states
- Outputs
- State transition logic
- Output logic

FSMs can be categorized based on whether outputs depend only on the present state (Moore) or present state and inputs (Mealy).

1.3 Mearly FSM Concept

In a Mealy finite state machine, the output depends on the present state and the input. A Mealy machine reacts faster to an input change than a Moore machine does since its output can change independently of a state change.

The reasons for the use of Mealy FSM is the following:

- Transitions between states are based on the DONE signals.
- Some outputs depend directly on these input signals.

- This greatly improves performance and responsiveness.

1.4 Description of the Production Process

Product-2 is manufactured on a sequence of machines in a fixed order:

M1 → M5 → M3 → M6 → M7

Each machine sends a DONE signal to show its completion and that the next machine can begin. The controller is initially in an idle state and proceeds through the sequence of machines one at a time.

After it is done with the last machine (M7), the automaton returns to its idle state. This automaton has been modeled as Mealy-type finite state machine using D flip-flops.

2 PROBLEM DEFINITION

The main object of this project is the design of a finite state machine-based control system for a production proces Product-2. The system provides machines to work in a spesific order. The next machine can't start until the previous one finished its work. The design was made utilizing a FSM of the the Mealy type.

2.1 Machine Order for Product-2

The production of Product-2 is being done in the following order:

M1 → M5 → M3 → M6 → M7

Machines operates in this order one-by-one.

Each machine must operate before the next one begins. The system does not skip any machine or change the order of processing. The system returns back to the idle state as soon as the last machine has completed that job and waits for a start signal.

2.2 Input Signals

FSM uses several input signals to control state passings. These signals show either the initial of the process or the completion of machine operations.

The input signals are defined as follows:

- P2: Start signal for Product-2
- D1 (DONE_M1): Indicates completion of Machine 5
- D5 (DONE_M5): Indicates completion of Machine 5
- D3 (DONE_M3): Indicates completion of Machine 3
- D6 (DONE_M6): Indicates completion of Machine 6
- D7 (DONE_M7): Indicates completion of Machine 7

The DONE signals are asserted or set to logic '1' upon the machines finishing processing their respective requests. The FSM uses these values for determining if it will remain in the present state or transition to another.

2.3 Output Signals

The outputs of the controller are used in order to activate machines during the production process. Each output corresponds to one machine.

The output signals are defined as follows:

- M1: Enables Machine 1
- M5: Enables Machine 5
- M3: Enables Machine 3
- M6: Enables Machine 6
- M7: Enables Machine 7
- FINISH: Shows that the entire production cycle is completed

Only one machine output is thus activated at any time. The signal FINISH is activated when the last machine M7 has completed its part of the job.

2.4 Design Constraints and Assumptions

In this project, these rules are applied in the design:

- The system begins in an idle state.
- The production order of the representative machines must be followed.
- No more than one machine is in use.
- D flip-flops are used to store the state information.
- When logic is minimized, unused states are considered don't care conditions.
- The controller is implemented under a Mealy finite state machine.
- The current state and the input signal or signals determine state transitions.
- The design must be implementable within MATLAB/Simulink and simulatable within MATLAB/Simulink.

3 STATE DEFINITIONS AND STATE DIAGRAM

In this part, the states that are used in the system and transitions between these states are explained. Each state represents a spesific step in the production process.

3.1 Definition of System States

The FSM consist of six main states and each state, corresponds one step in the production sequence or the idle condition.

- S0 – Idle State:
  The system wait for the start signal P2. No machine is active in this state.
- S1 – Machine M1 Active:
  Machine M1 is enabled and begins processing the product.
- S2 – Machine M5 Active:
  After M1 finished, the system activates machine M5.
- S3 – Machine M3 Active:
  After M5 finished, the system activates machine M3.
- S4 – Machine M6 Active:
  After M3 finished, the system activates machine M6.
- S5 – Machine M7 Active:
  After M1 finished, the system activates machine M7. When M7 is completed, the system returns to the idle state.

3.2 Description of Each State

Each state represents a unique condition of the system as follows:

- S0 (Idle): Waiting state. The controller wait for the begin signal P2.
- S1: Machine M1 is active until DONE_M1 becomes 1.
- S1: Machine M5 is active until DONE_M5 becomes 1.
- S1: Machine M3 is active until DONE_M3 becomes 1.
- S1: Machine M6 is active until DONE_M6 becomes 1.
- S1: Machine M7 is active until DONE_M7 value is 1, after which the process ends.

In the system, only one machine can operate at the same time and the system passes the next state only the DONE signal is received.

3.3 State Transition Explanation

State transitions are controlled by input signals and follow a fixed sequence. Transitions occur as follows:

- Fron S0 to S1 when the start signal P2=1
- From S1 to S2 when DONE_M1=1
- From S2 to S3 when DONE_M5=1
- From S3 to S4 when DONE_M3=1
- From S4 to S5 when DONE_M6=1
- From S5 to S0 when DONE_M7=1

If the required input signal isn't active, the FSM remains in the current state.

3.4 State Diagram (Mealy FSM)

The state diagram of the system is a Mealy FSM, meaning that the next state depends both on the current state and upon the input signals to the system. The state diagram transitions are labeled with the conditions that cause the transitions.

- States are represented as circles (S0 - S5).
- Arrows show the transition between states.
- Each arrow has a label for the condition when the transition occurs.
- If this condition satisfies not, the FSM remains in its current state.

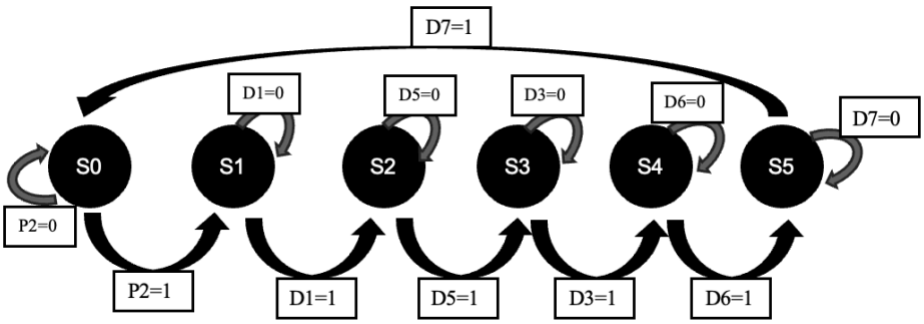

Figure 3.1 State diagram of the Mearly FSM for Product-2

The state diagram of the designed Mealy finite state machine is shown in *Figure 3.1*.

A state represents a step in a production procedure, and states transition only with the correct input. If the input signal does not satisfy a condition, the system changes not in its state. The next diagram shows the order of the machines that someone processes and someone transitions to the idle state after the last machine processes.

4 STATE ENCODING

In this section, how the states in the system are encoded in binary form is explained. The states are represented using flip-flops.

4.1 Selection of Number of Flip-Flops

The system consists of 6 distinct states (S0-S5). For representing these states in binary form, sufficient count of flip-flops must be selected and this number of flip-flops is calculated as follows:

$$2^n \geq Number\ of\ states$$

Since there are 6 states in this system:

$$2^2 = 4 < 6, \qquad 2^3 = 8 \geq 6$$

For this reason, to encode all system states, three flip-flops are reequired.

These three flip-flops are labeled as Q_2, Q_1 and Q_0.

4.2 Binary Encoding States

Each of the states are assign a unique 3-bit binary code as shown in the *Table 4.1*.

Table 4.1 State Encoding of the FSM

State	Description	$\mathbf{Q_2\ Q_1\ Q_0}$
S0	Idle state	000
S1	M1 active	001
S2	M5 active	010
S3	M3 active	011
S4	M6 active	100
S5	M7 active	101

The encoding in the *Table 4.1* helps to represent the system states simplier and systematic.

4.3 Unused States

With the usage of 3 flip-flops, 8 distinct situation occurs. However, only 6 of them are used. The remaining 2 binary combinations are unused:

- 110
- 111

The unused states are treated as don't care conditions during the logic minimization to minimize the Boolean equations of the next-state logic and output logic.

5 NEXT STATE (TRANSITION) TABLE

5.1 Construction of the Transition Table

The next state (transition) table describes how the next state of the finite state machine will be computed based on the present state and the input signals. The next state table can be generated via considering all possible values of the state bits together with all the possible inputs to the FSM.

The transition table for the system can be obtained by considering only those input conditions that are relevant at each state because only one of the inputs is meaningful at any one state in the system. This allows the design toward simplification without altering the correctness of the FSM.

5.2 Input Conditions for State Transitions

For each state, the FSM checks only the input signal of the currently active machine. Then upon this signal being logic '1', the FSM passes to the next state. The FSM stays in its current state if the input signal is a logic '0'.

All other input signals that do not influence the transition in the specific state are treated as don't care (X) conditions. This reduction strategy is recursively applied for each layer of the logic design. It is used in the logic minimizations discussed in *Sections 6 and 7*.

5.3 Final Next-State Table

The reduced next state table of the FSM is shown in *Table 5.1* and the table defines complete state transition behavior of the system.

Table 5.1 Next-State Table

Present State ($Q_2Q_1Q_0$)	P2	D1	D5	D3	D6	D7	Next State ($Q_2^+Q_1^+Q_0^+$)
000 (S0)	0	X	X	X	X	X	000
000 (S0)	1	X	X	X	X	X	001
001 (S1)	X	0	X	X	X	X	001
001 (S1)	X	1	X	X	X	X	010
010 (S2)	X	X	0	X	X	X	010
010 (S2)	X	X	1	X	X	X	011
011 (S3)	X	X	X	0	X	X	011
011 (S3)	X	X	X	1	X	X	100
100 (S4)	X	X	X	X	0	X	100
100 (S4)	X	X	X	X	1	X	101
101 (S5)	X	X	X	X	X	0	101
101 (S5)	X	X	X	X	X	1	000

6 FLIP-FLOP SELECTION AND EXCITATION TABLE

6.1 Selection of D Flip-Flop

In this project, D flip-flops are used to implement the state elements of the finite state machine because they are efficient and simple to use, and have the added benefit of the input being related directly to the next state.

For a D flip-flop, the next state of the output is given by:

$$Q^+ = D$$

This property allows the next-state values from the transition table to be wired directly to the D inputs of the flip-flops, resulting a simpler design with more reliability.

6.2 Relation Between Next State and D Inputs

Because of the D flip-flops usage, the excitation inputs of the flip-flops are the same as their next-state values. Thus, the excitation equations can be written directly from the next state table.

For the 3 state bits Q_2, Q_1 and Q_0, the following relations apply:

- $D_2 = Q_2^+$
- $D_1 = Q_1^+$
- $D_0 = Q_0^+$

This direct relationship eliminates the need for a separate excitation conversion table, simplifying the design.

6.3 Final Excitation Table

Based on the next state table *Table 5.1* shown in *Section 5*, excitation table is made by assigning the next-state bits directly to the D inputs. The final excitation table is given in *Table 6.1*.

Table 6.1 Excitation Tablo of the FSM

Present State ($Q_2Q_1Q_0$)	Input condition	Next State ($Q_2^+Q_1^+Q_0^+$)	D_2	D_1	D_0
000	P2 = 0	000	0	0	0
000	P2 = 1	001	0	0	1
001	D1 = 0	001	0	0	1
001	D1 = 1	010	0	1	0
010	D5 = 0	010	0	1	0
010	D5 = 1	011	0	1	1
011	D3 = 0	011	0	1	1
011	D3 = 1	100	1	0	0
100	D6 = 0	100	1	0	0
100	D6 = 1	101	1	0	1
101	D7 = 0	101	1	0	1
101	D7 = 1	000	0	0	0
110	–	XXX	X	X	X
111	–	XXX	X	X	X

7 MINIMIZATION OF NEXT-STATE LOGIC

In this project, a finite state machine (FSM) is created using D-type flip-flops, so the next-state logic is defined by the D flip-flop inputs, which are D2, D1, and D0. The next state (D) depends on the present state (Q), represented by Q1, Q2, and Q0, and the DONE signals of the machines. According to this, Boolean expressions for the D inputs are obtained from the next-state table.

7.1 Karnaugh Map Method

The Karnaugh Map (K-map) method is used to simplify Boolean expressions that are obtained from the next-state table. In this project, K-maps reduce the number of gates used in the project by simplifying the next-state expressions. Since the FSM is created with D-type flip-flops, separate Karnaugh maps were created for each D input. The present state bits Q2, Q1 and Q0 are used as the map variables, and unused state combinations are accepted as “don’t care” conditions. In Karnaugh maps, by grouping (2, 4, 8 or 16), simplified Boolean expressions are obtained for the next-state logic. These simplified expressions make the overall circuit easier to understand and implement.

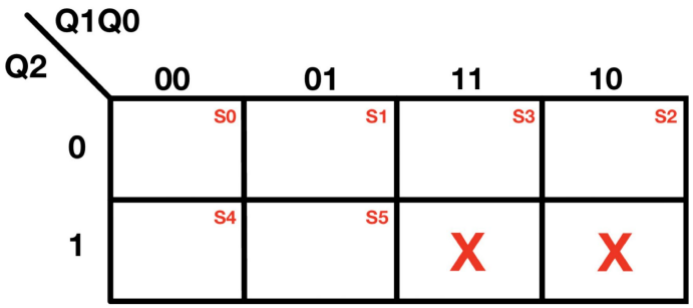

Figure 7.1 Structure of K-map

The Karnaugh Map shown in the *Figure 7.1* was implemented using present state variable Q2, Q1 and Q0. Each cell in the map represents the states which are S0(IDLE), S1, S2, S3, S4, and S5, and are labelled in red in the figure. In the project, 3 flip-flops were used which means that there are total of 8 states that can be used. Since only 6 state was required, two cells in the map marked as “X” meaning “don’t care” conditions. These don’t care cells are included to allow further simplification of the logic expressions.

7.2 Minimization of D_2 Logic

In the k-map created for D2, *Figure 7.2*, values are given considering both the present state and the next state. For cell S1, when DONE_M1=0, it must remain in the present state, which is 001, and when DONE_M1=1, it must transition to the next state, S2 state, which is 010. In both cases, Q2 remains 0 and does not change, so 0 is written to cell S1. Following the same logic, the values for cells S0 and S2 are also written as 0, as shown in the figure.

For cell S4, the value is determined by looking at the present state and next state, similar to the previous states. For S4, when the DONE input is 0, it remains in the present state, stays at 100; when it is 1, it transitions to the next state, transitions to S5 (101). In both cases, since the Q2 value does not change, 1 is written to the S4 cell.

For the S3 cell, the same method is used by providing the DONE input to look at the Q2 states of the states that are passed or remained in. When DONE_M3=0 in S3, this means remaining in the present state, S3, which is 011, while when DONE_M3=1, it means moving to the next state, S4, which is 100. In both cases, when we look at the Q2 value, we observe that the value changes depending on the DONE input. When DONE_M3=0, the Q2 value is also 0, and when DONE_M3=1, the Q2 value is also 1, so D3 is given as the DONE input to cell S3.

The logic applied to S3 and other cells is applied in the same way to the S5 cell. Depending on the DONE input, S5 either remains in the present state (101) or transitions to the next state (000). When DONE_M7=0, the Q2 value is 1, while when DONE_M7=1, the Q2 value is 0. In this case, as they are opposites of each other, D7 is written to the S5 cell. Since 110 and 111 are unused cases, they remain as don't care.

Q2 \ Q1Q0	00	01	11	10
0	S0: 0	S1: 0	S3: D3	S2: 0
1	S4: 1	S5: $\overline{D7}$	X	X

Figure 7.2 Karnaugh Map for D2

When creating an expression from this k-map, values of 0 and don't care are not used. This is because the expression written in the k-map cells is only valid when that state is active. In this map, it is written using S3, S4, and S5. The mathematically derived expression is written as:

$$D2 = (S3 \,.\, D3) + (S4 \,.\, 1) + (S5 \,.\, \neg D7)$$

If the bit equivalents of the states in this expression are written explicitly as S3 = Q2'Q1Q0, S4 = Q2Q1'Q0', and S5 = Q2Q1'Q0, then the expression at hand is obtained as:

$$D2 = (Q2'Q1Q0 \,.\, D3) + (Q2Q1'Q0') + (Q2Q1'Q0 \,.\, \neg D7)$$

The final expression is written as the sum of products of the active states and their associated conditions.

7.3 Minimization of D_1 Logic

The logic for creating the k-map for D1 proceeds as described in the previous section. States S0, S4, and S5 remain in the present state or transition to the next state based on the 1s and 0s they receive from the DONE signal. For the S4 cell, when DONE_M6=0, S4 must remain in the present state, which is 100, and when DONE_M6=1, it must transition to the next state, the S5 state, which is 101. In both of these cases, Q1 remains 0 and does not change, so 0 is written to the S4 cell. The S0 and S5 states are filled with the value 0 as specified in the logic diagram.

In cell S2, the value is determined by looking at the present state and next state, similar to the previous states. For S2, when the DONE signal is 0, it remains in the present state, which is 010, and when it is 1, it transitions to the next state, which is S3 (011). In both cases, since the Q2 value does not change, 1 is written to cell S2.

When DONE_M1=0 in S1, this means it remains in the present state, S1(001), while when DONE_M1=1, it means it transitions to the next state, S2(010). In both cases, looking at the Q1 value, we observe that the value changes depending on the DONE signal. When DONE_M1=0, the Q1 value is also 0, and when DONE_M1=1, the Q1 value is also 1, so D1 is given as the DONE signal to cell S1.

The logic that applied to S1 and to other cells is applied in the same way to cell S3. Depending on the DONE signal, S3 either remains in the present state (011) or transitions to

the next state (100). When DONE_M3=0, the Q1 value is 1, while when DONE_M3=1, the Q1 value is 0. In this case, as they are opposites of each other, D3 is written to the S3 cell. Since 110 and 111 are unused cases, they remain as don't care.

Q2 \ Q1Q0	00	01	11	10
0	S0: 0	S1: D1	S3: $\overline{D3}$	S2: 1
1	S4: 0	S5: 0	X	X

Figure 7.3 Karnaugh Map for D1

When creating an expression from this k-map, values of 0 and don't care are not used. This is because the expression written in the k-map cells is only valid when that state is active. In this map, it is written using S1, S2, and S3. The mathematically derived expression is written as:

$$D1 = (S1 \,.\, D1) + (S3 \,.\, \neg D3) + (S2 \,.\, 1)$$

If the bit equivalents of the states in this expression are written explicitly as S1 = Q2’Q1’Q0, S2 = Q2’Q1Q0’, and S3 = Q2’Q1Q0, then the expression at hand is obtained as:

$$D1 = (Q2'Q1'Q0 \,.\, D1) + (Q2'Q1Q0 \,.\, \neg D3) + (Q2'Q1Q0')$$

7.4 Minimization of D0 Logic

The same logic used in the other two cases is also used when creating the k-map for D0. Here, unlike the other sections, no cell contains a 0 or 1 as shown in the diagram. Let's start with S0. Here, again unlike the cells in the other sections, there is no DONE signal, but there is a start signal (P2). When P2=0, it means stay in the present state, which is 000, when P2=1, it means move to the next state, S1, which is 001. Here, since Q0 changes in both cases depending on P2, P2 is written to the S0 cell.

When DONE_M1=0 arrives at the S1 cell, it remains in the present state, S1(001). When DONE_M1=1 arrives, it transitions to the next state, S2(010). Since the Q0 value is 1 when DONE_M1=0 and 0 when DONE_M1=1, it is inverted, and the inverse of D1, ¬D1, is written

to the S1 cell. The same inverse logic is applied to the S3 and S5 cells, writing ¬D3 and ¬D7 respectively, as shown in the diagram.

The same logic used in S1, S3, and S7 is applied in cell S2, only without taking the inverse. When DONE_M5=0, stay in the present state, which is 010, in S2, when DONE_M5=1, move to the next state, which is S3, which is 011. Here, since the Q0 value is 0 when DONE_M5=0 and 1 when DONE_M5=1, it is connected to D5, and D5 is written to cell S2. The same logic is applied to cell S4, and D6 is written as shown in the diagram.

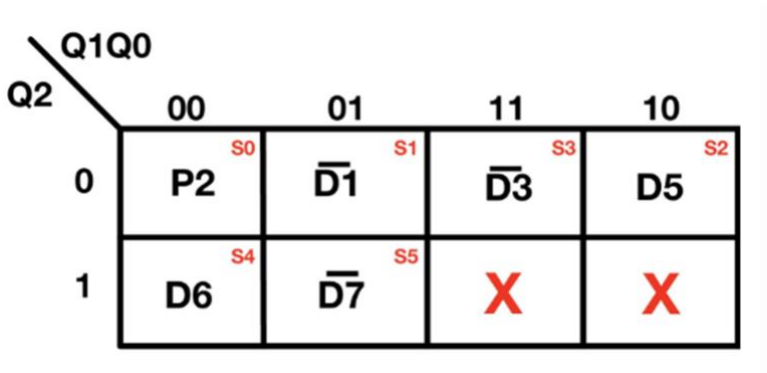

Figure 7.4 Karnaugh Map for D0

When forming the expression from this k-map, the values 0 are not used. This is because the expression written in the k-map cells is only valid when that state is active. In this map, it is written using S1, S2, S3, S4 and S5. The logically derived expression is written as:

D0 = (S0 . P2) + (S1 . ¬D1) + (S2 . D5) + (S3 . ¬D3) + (S4 . D6) + (S5 . ¬D7)

If we explicitly write the bit equivalents of the states in this expression as S0 = Q2'Q1'Q0', S1 = Q2'Q1'Q0, S2 = Q2'Q1Q0', S3 = Q2'Q1Q0, S4 = Q2Q1'Q0', and S5 = Q2Q1'Q0, then the expression becomes:

D0 = (Q2'Q1'Q0' . P2) + (Q2'Q1'Q0 . ¬D1) + (Q2'Q1Q0' . D5) + (Q2'Q1Q0 . ¬D3) + (Q2Q1'Q0' . D6) + (Q2Q1'Q0 . ¬D7)

Unused states, which are don't cares, allow simplification. However, in the D0 k-map, most cells are variables (P2, ¬D1, D5…), so the classic method of grouping and simplifying 1s is limited.

7.5 Summary of Minimized Next-State Equations

This section summarizes the minimized Boolean equations for the FSM next-state logic equations. From the minimization equations, the D-type flip-flop input equations D2, D1, D0 are derived from the present state bits and the associated DONE signals using the Karnaugh map minimization technique. For some states, next-state bits are constants, while in other states these bits depend on the value of the related input signals. Unused state combinations were accepted as don’t care conditions to simplify the logic. These final minimized equations can be used directly to implement the next-state behavior of the FSM circuit.

8 OUTPUT TABLE AND OUTPUT LOGIC

8.1 Definition of Output Signals

The output signals of the FSM control the machines in the production process, one output for each machine, indicating whether the machine is running or has been turned off by the FSM.

The output signals are defined as follows:

- M1: Activates Machine 1
- M5: Activates Machine 5
- M3: Activates Machine 3
- M6: Activates Machine 6
- M7: Activates Machine 7

FINISH: Indicates the end of the entire production process

Only one machine output can activated at the same time for ensuring sequential operation.

8.2 Mealy Output Behavior

In Mealy-type finite state machines, the outputs may depend not only on the current state of the FSM, but also on the input signals.

In the design, most machine control outputs (M1-M7) are determined only from the current state, so this could be modeled as a Moore FSM. The FINISH output is dependent on the current state and the value of the DONE_M7 input signal, so this is a Mealy FSM.

Specifically, the FINISH signal is active if the FSM is in S5 and the input signal DONE_M7 is equal to 1.

8.3 Output Table

The output table lists output values of the FSM in each state for the corresponding input. In this case, the FSM output value in each state is shown in *Table 8.1*.

Table 8.1 Output Table of the FSM

Present State	$Q_2Q_1Q_0$	M1	M5	M3	M6	M7	FINISH
S0 (IDLE)	000	0	0	0	0	0	0
S1 (M1)	001	1	0	0	0	0	0
S2 (M5)	010	0	1	0	0	0	0
S3 (M3)	011	0	0	1	0	0	0
S4 (M6)	100	0	0	0	1	0	0
S5 (M7)	101	0	0	0	0	1	D7 (DONE_M7)

8.4 Minimization of Output Logic

The output table is used to obtain the boolean expressions for the output signals. In most cases, the output signals depend on the state of the system. Because of this, their logic expressions are simple.

Machine control outputs are expressed as:

- M1=S1
- M5=S2
- M3=S3
- M6=S4
- M7=S5

In addition to these, the "FINISH" output depends on both the input signal and the state:

- FINISH=S5.D7

The FINISH output can be active only when the system is in state S5 and the DONE_M7 input is equal to 1. The FINISH output remains inactive for all other state and input combinations.

Karnaugh maps are often used to simplify Boolean expressions, but not all logic functions can be simplified further. Due to this restriction of the design, certain output functions are defined by a single minterm and cannot be grouped in the Karnaugh map.

This shows that these forms of expressions are already minimized, and because of this, they cannot be reduced further. An example of a Karnaugh map of the FINISH output signal is shown in *Table 8.2*.

The Karnaugh map for the FINISH output shown in the *Table 8.2*, contains only one logic '1' and all others are logic '0's. No grouping can be performed because there are no any adjacent '1' cells. Thus, the expression is already minimal.

Table 8.2 Example Karnough Map for FINISH output

S5 \ D7	**0**	**1**
0	0	0
1	0	**1**

Table 8.2 confirms that the FINISH=S5.D7 can't be further simplified.

In addition, the machine control outputs M1, M5, M3, M6 and M7 depend only on the present state of the FSM. In each of these outputs, the output is enabled from one state only and disabled in all other states. Therefore, these outputs are single minterms each in their minimized form.

9 LOGIC CIRCUIT DESIGN

The logic circuit implementation of the designed finite state machine is presented next. In this section, the FSM is implemented with logic gates and D flip-flops using the minimal next-state and output equations derived in the previous sections. The complete logic diagram for the overall system includes the next-state logic circuit, the output logic circuit, and the state register.

9.1 Overall System Block Diagram

This block-level view diagram gives an overview of the structure of an FSM-based control system. It illustrates the basic logic building blocks of the system and how the inputs, next state logic, state register and output logic are interrelated. The gate-level details are not indicated in this diagram.

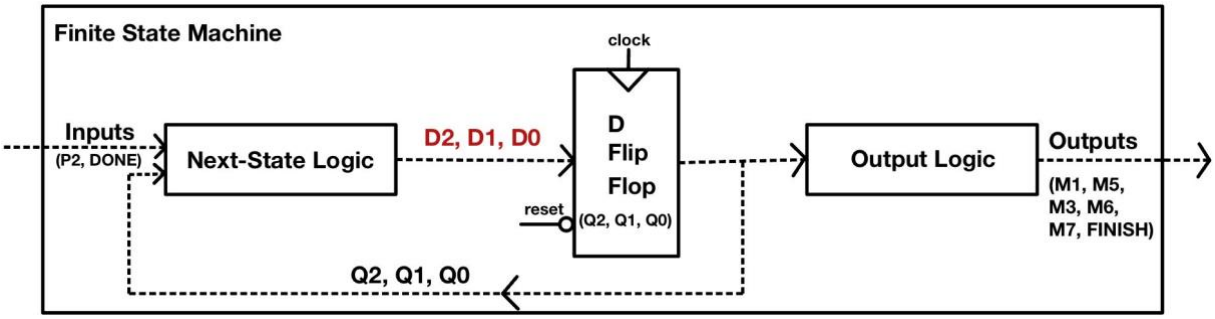

Figure 9.1 FSM in Block Diagram

The next state logic generates the next state output value based on the input signals as shown in the block diagram. The present state information is held in a D flip-flop register which latches the next state output values provided by the next state logic shown in the block diagram. This state information is fed back into the next-state logic and is also used by the output logic to determine the control signals.

9.2 Next-State Logic Circuit

In this section, the next-state logic circuit of the FSM is presented. The circuit is implemented directly from the section 7, the obtained expressions were used.

9.2.1 Logic Diagram of D2

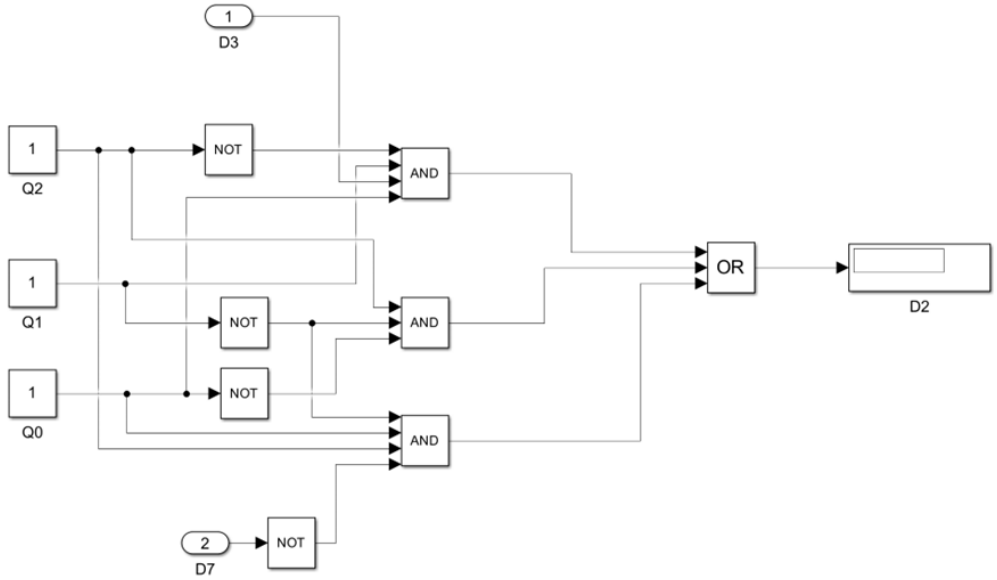

Figure 9.2 Simulink model of Next-State Diagram of D2

The expression D2 = (Q2'Q1Q0 . D3) + (Q2Q1'Q0') + (Q2Q1'Q0 . ¬D7) derived in Section 7 has been implemented using MATLAB Simulink as shown in *Figure 9.2.* Separate Simulink models were created for each set of parentheses in the expression, making them easier to understand, as shown in *Figures 9.3, 9.4,* and *9.5.*

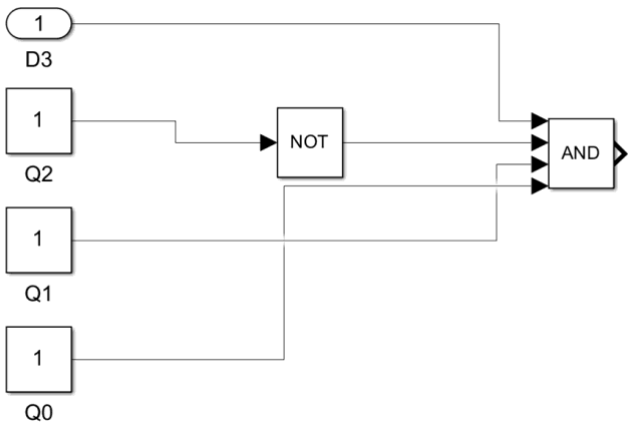

Figure 9.3 Simulink model of (Q2'Q1Q0 . D3)

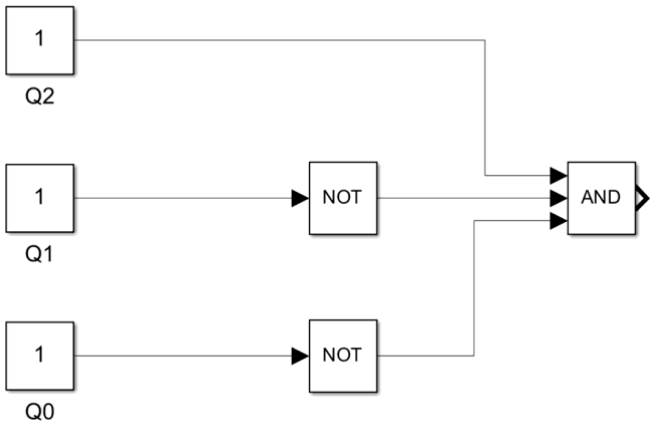

Figure 9.4 Simulink model of (Q2Q1'Q0')

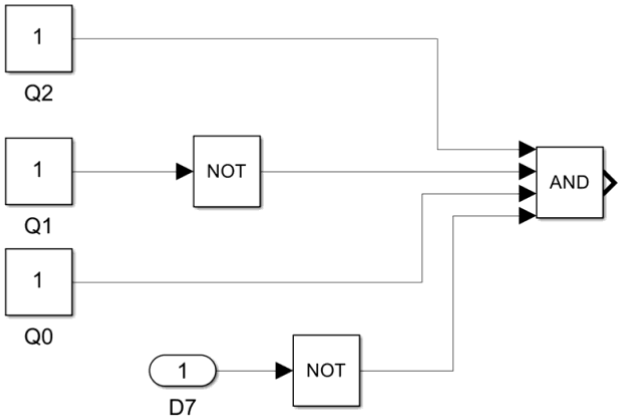

Figure 9.5 Simulink model of (Q2Q1'Q0 . ¬D7)

9.2.2 Logic Diagram of D1

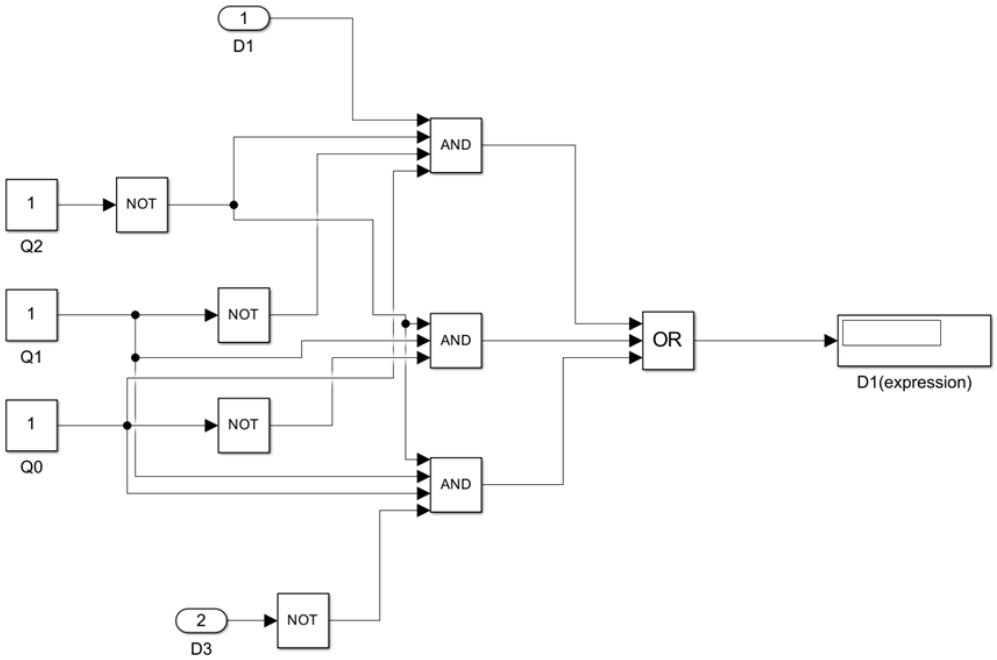

Figure 9.6 Simulink model of Next-State Diagram of D1

The expression D1 = (Q2'Q1'Q0 . D1) + (Q2'Q1Q0 . ¬D3) + (Q2'Q1Q0') derived in Section 7 has been implemented using MATLAB Simulink as shown in *Figure 9.6.* Separate Simulink models were created for each set of parentheses in the expression, making them easier to understand, as shown in *Figures 9.7, 9.8,* and *9.9.*

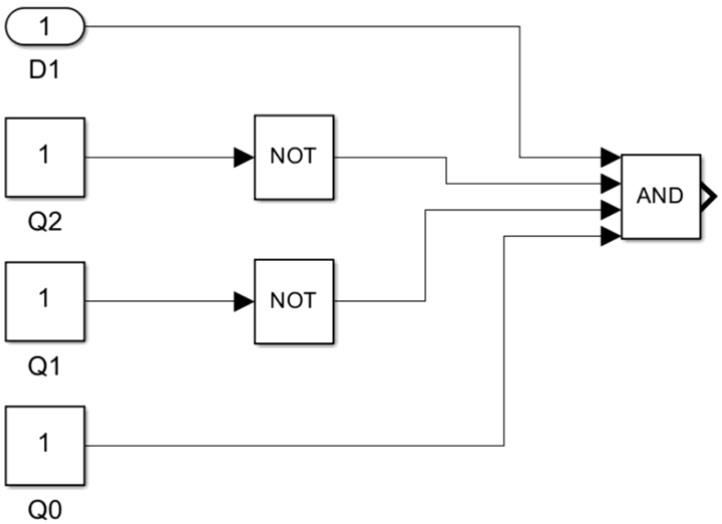

Figure 9.7 Simulink model of (Q2’Q1’Q0 . D1)

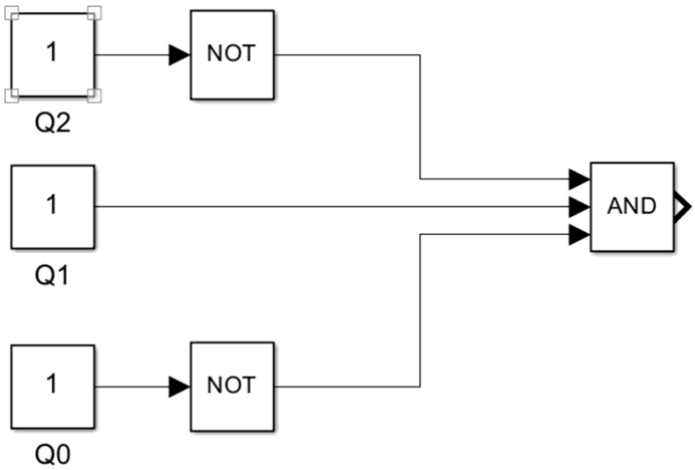

Figure 9.8 Simulink model of (Q2'Q1Q0')

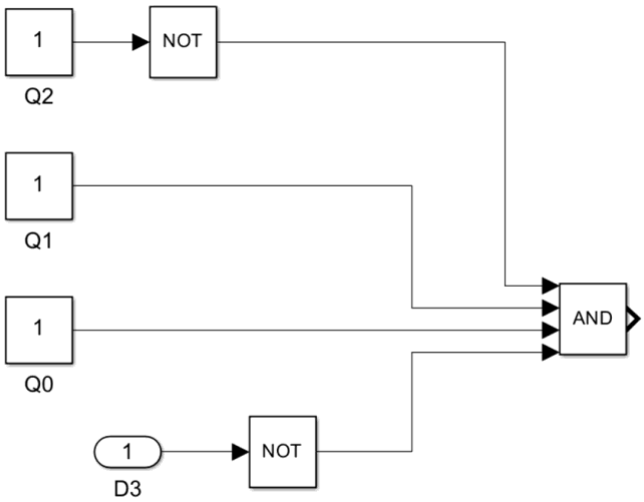

Figure 9.9 Simulink model of (Q2'Q1Q0 . ¬D3)

9.2.3 Logic Diagram of D0

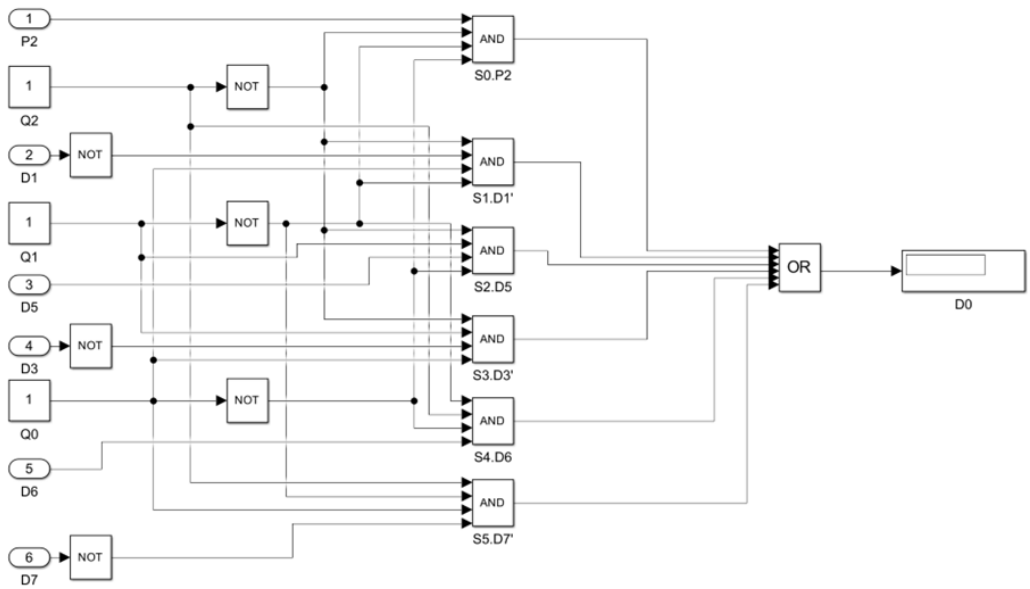

Figure 9.10 Simulink model of Next-State Diagram of D0

The expression D0 = (Q2'Q1'Q0' . P2) + (Q2'Q1'Q0 . ¬D1) + (Q2'Q1Q0' . D5) + (Q2'Q1Q0 . ¬D3) + (Q2Q1'Q0' . D6) + (Q2Q1'Q0 . ¬D7) derived in Section 7 has been implemented using MATLAB Simulink as shown in *Figure 9.10*. Separate Simulink models were created for each set of parentheses in the expression, making them easier to understand, as shown in *Figures 9.11, 9.12, 9.13, 9.14, 9.15* and *9.16.*

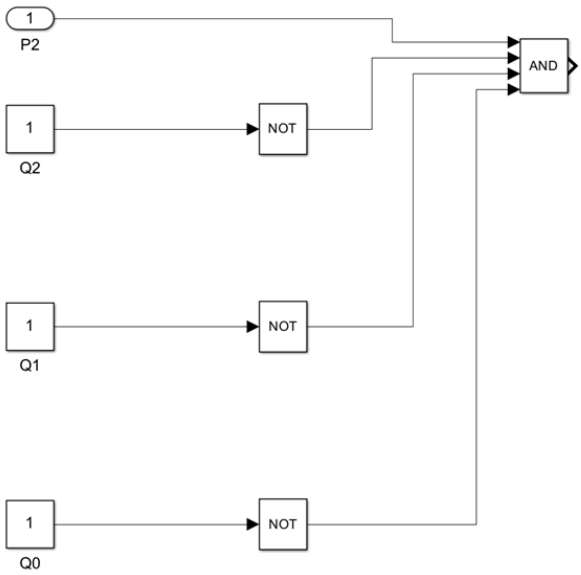

Figure 9.11 Simulink model of (Q2'Q1'Q0' . P2)

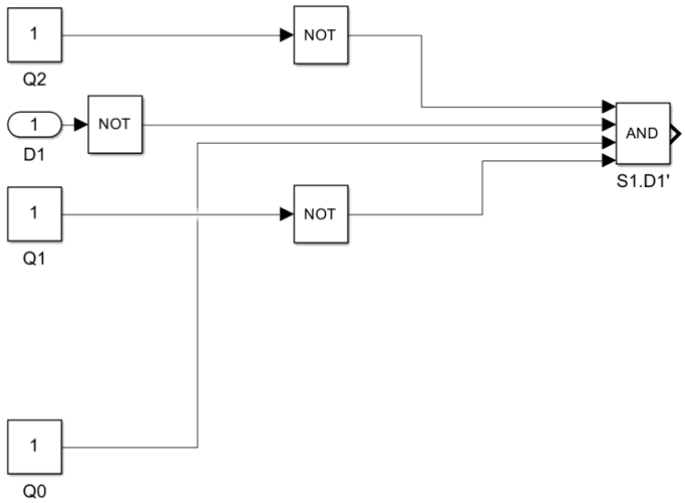

Figure 9.12 Simulink model of (Q2'Q1'Q0 . ¬ D1)

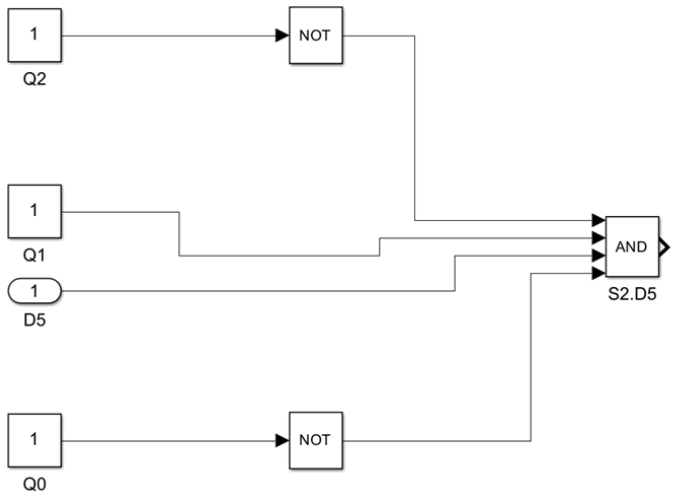

Figure 9.13 Simulink model of (Q2'Q1Q0' . D5)

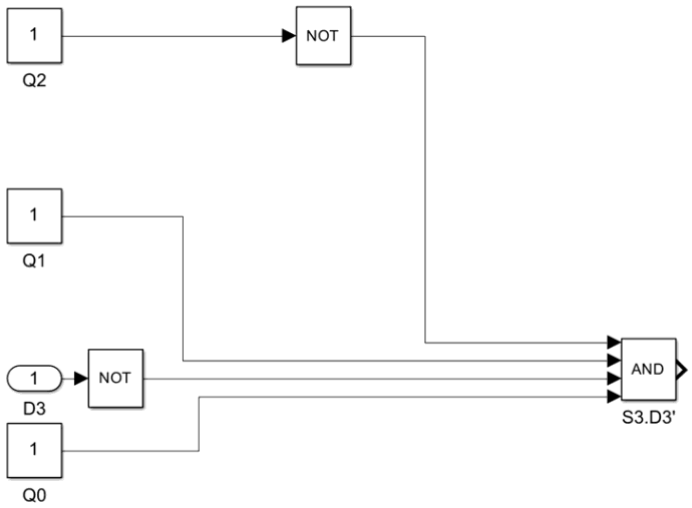

Figure 9.14 Simulink model of (Q2'Q1Q0 . ¬D3)

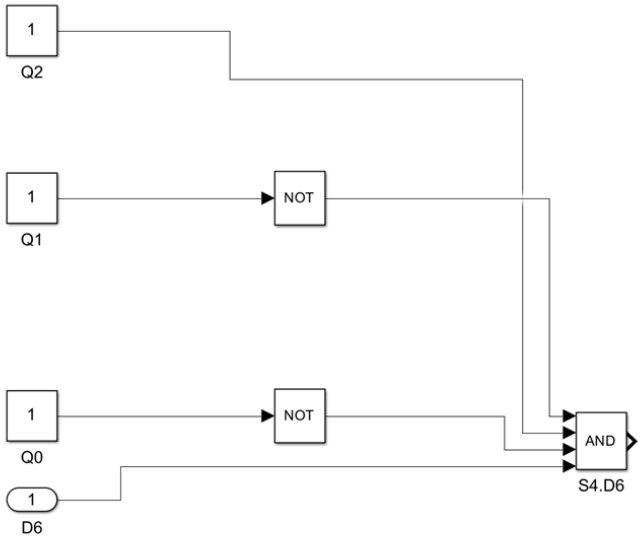

Figure 9.15 Simululink model of (Q2Q1'Q0' . D6)

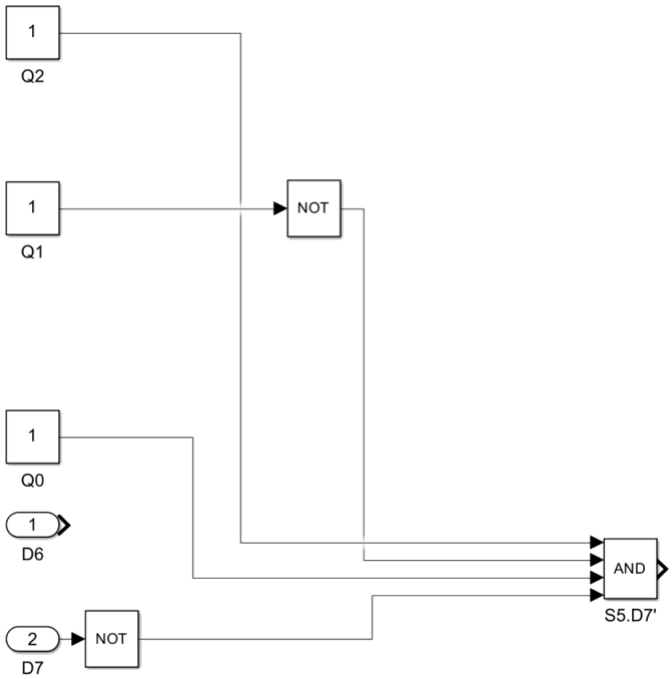

Figure 9.16 Simulink model of (Q2Q1'Q0 . ¬D7)

9.3 Output Logic Circuit

The output logic circuit of the FSM is designed by the Boolean expressions that obtain in Section 8. The aim of this circuit is to generate the machine control signals and the FINISH signal depending on the state of the FSM and the input signal for Mealy output machines.

For this design, machine control outputs M1, M5, M3, M6 and M7 are functions only of the present state of the FSM and can be generated by means of decoding the state variables Q_2, Q_1, Q_0. Each state signal is generated by combinational logic, such that the protocol state signals correspond to the binary code for the state S5 is generated when $Q_2 = 1$, $Q_1 = 0$ and $Q_0 = 1$.

The decoding logic functions are expressed in NOT and AND gate form to detect the required combinations of state variables. The decoded state variable is directly wired to the related machine output, ensuring that only a single machine can be in operation at once, consistent with the sequential nature of the production process.

The FINISH output is generated quite differently. Because the modeled FSM is a Mealy-type FSM, the FINISH signal depends both on the current state and the input signal. The FINISH signal is generated when both the decoded state S5 and the input signal DONE_M7 (D7) are true, confirming the Mealy behavior of the FSM, that is, the output signal is dependent on the state and the input signal.

The complete output logic circuit was implemented in MATLAB/Simulink blocks as a schematic designing tool, shown in the *Figure 9.17*. Simulink is used only to show the logical structre of the circuit. The functional simulation and verification, though, is described in *Section 10*.

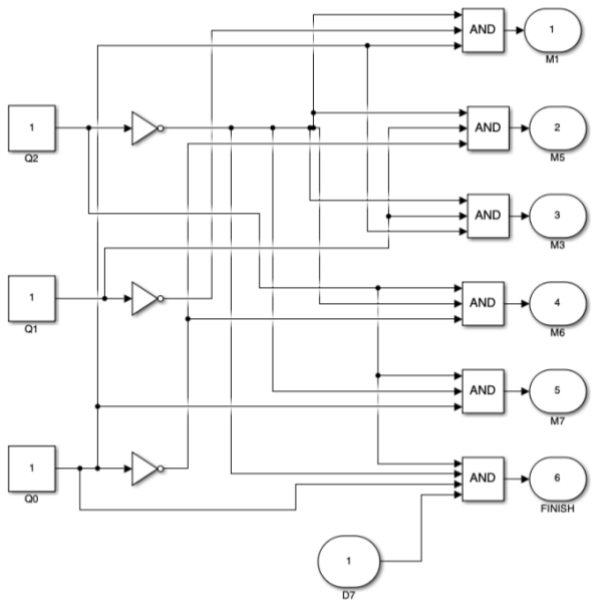

Figure 9.17 Output logic circuit implemented using Simulink blocks

Figure 9.17 illustrates the decoding of the state variables and the generation of the machine control outputs and the FINISH signal.

9.4 D Flip-Flop Register and Initialization

In this subsection we describe the state register of the finite state machine, its register structure, its clocking and how the reset signal initializes it. As mentioned in the previous sections, a D flip-flop can be used to implement the FSM. The value of the next state is simply output on the active edge of the clock, represented by Q(next)=D. In the circuit, three D flip-flops are used to implement the state register storing the state of the FSM in this mode. One flip-flop is assigned to each state bit of the FSM generating Q2, Q1 and Q0 which are the present state of the system.

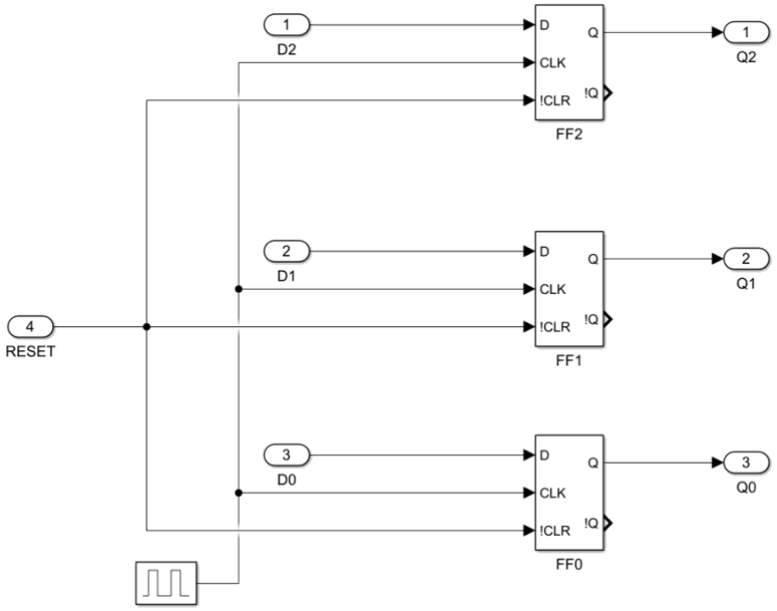

Figure 9.18 Simulink model of Simulink model of D flip-flop state register

In *Figure 9.18*, the D flip-flop register is shown, it is used in the FSM implementation. The next-state logic supplies the D inputs by the next-state signals D2, D1 and D0, which are computed. A reset input sets the register to the zero state S0(Q2Q1Q0=000) by applying a common clock signal to all of the flip-flops. S0 is the zero state so the system starts in a known state from there. The next-state logic section drives the new state of the FSM, while the register holds the current state.

9.5 Complete Logic Diagram

The complete FSM circuit diagram will use the minimized next-state logic, output logic, and D flip-flop register described in previous subsections. The completed diagram gives a

complete picture of how the machine works, and will show all of the component functional blocks.

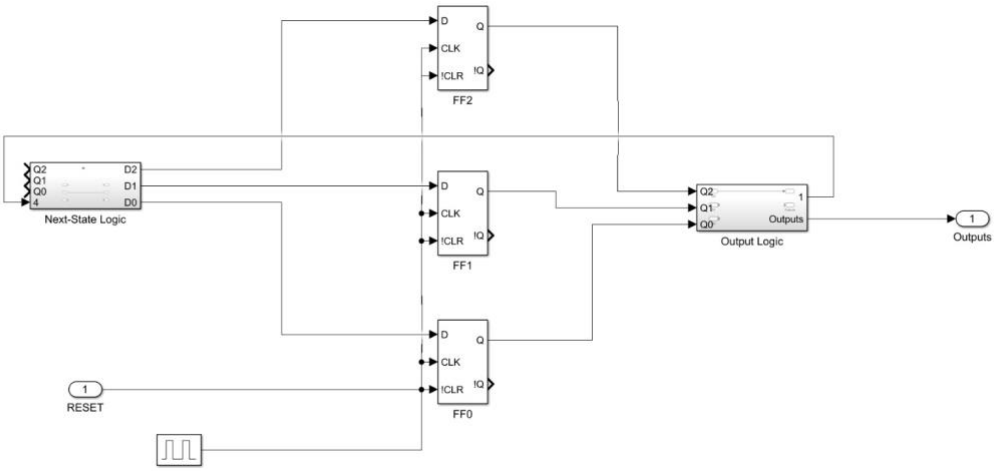

Figure 9.19 Complete FSM Diagram showing Next-State logic, D Flip-Flop Register and Output Logic in Simulink

The next-state logic, shown in the complete logic diagram, generates the next-state of the FSM as a function of its current state and input signals. The current state of the FSM is stored in a D flip-flop register and is clocked by the clock signal. The control outputs are generated by output logic based on the present state and other input signals.

10 SIMULATION AND VERIFICATION

10.1 MATLAB/Simulink Model Description

In this project, we developed a PLC based sequential control system using MATLAB/Simulink and Stateflow that looks to represent the theoretical FSM (Finite State Machine) design in the form of a digital clocked simulation of the final product. For theoretical description, we designed the system as a clocked synchronous sequential circuit having states, inputs and outputs, and memory. Conversely, in the simulation, we modeled the same set-up by specifying the block architecture, a PLC controller (a Stateflow chart) and other machine subsystems (Machine_M1, Machine_M5, Machine_M3, Machine_M6, Machine_M7).

The PLC block is defined by the Stateflow chart which is used as main FSM controller. In our theoretical design we described states IDLE, machine operation states M1, M5, M3, M6, and M7. We use states in Stateflow to model the same states we used in Simulink, such as the theoretical "machine 1 is active" state becoming the Stateflow state M1, and so on. The transition logic (next state logic) is encoded as the transition labels between the Stateflow states.

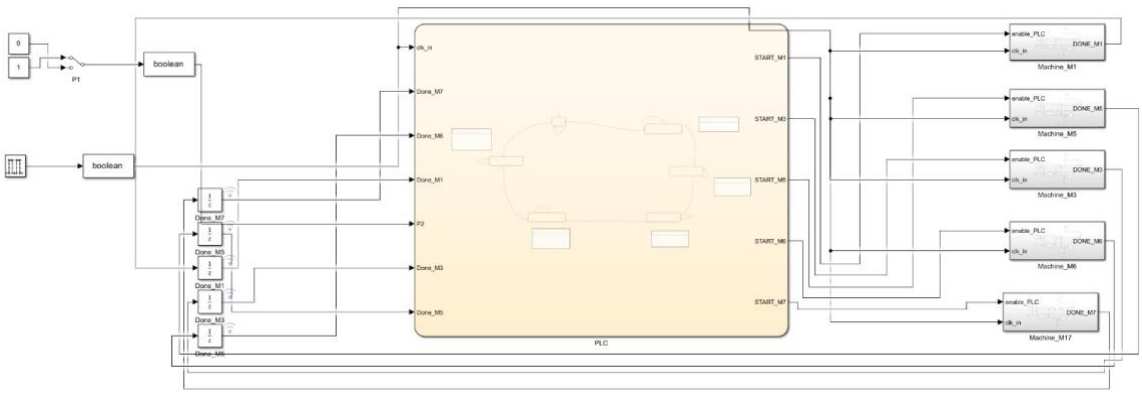

Figure 10.1 Overall Simulink system implementation

The PLC interacts with machines using two main types of signals:

10.1.1 START Signals (outputs of the PLC)

These are control outputs for the machine subsystem, shown in our model as control signals:

- START_M1

- START_M5
- START_M3
- START_M6
- START_M7

These theoretically correspond to the FSM output signals indicating the currently active operation. In the simulation, they are realized as Stateflow output data (scope = Output). Outputs are driven to the machine subsystems based on their input called enable_PLC (or machine enable input).

10.1.2 DONE Signals (inputs of the PLC)

Signals that a machine has completed its current task:

- Done_M1
- Done_M5
- Done_M3
- Done_M6
- Done_M7

In a theoretical FSM, these signals correspond to the "completion conditions" present in the transition table (for example, a transition fired if "Done_M1 = 1 then go to next state" in a theoretical FSM). For simulation purposes, the completion conditions are Stateflow input data (scope = Input) and come from the output ports of the machine subsystems. So: the theoretical completion flag is a boolean DONE output from each machine subsystem.

All of the above assumptions require that some clock input (clk_in) be shared between PLC and machine subsystems logic, and that some flip-flops and a clocked update rule will be used to model the state memory. This assumption is also exactly what the simulation satisfies as "synchronous update". State transitions and internal counters are updated on each clock signal tick.

Finally, we had to think about memory and timing behavior inside machines. In theory a machine is not instantaneous; it takes time/cycles. The time within the simulation, however, is modeled by a counter-like behavior using Unit Delay (1/z) blocks and other arithmetic operations, therefore, the machine process duration is equal to "N clock cycles" in the machine subsystem.

10.2 Simulation Setup

The simulation we are going to build should be as close as possible to the actual implementation, key point being that our system is clock-driven discrete-time (not continuous-time). The clock and memory blocks are critical in this scenario.

10.2.1 Solver Discrete-Time Operation

We ran the simulation in discrete mode because signals are updated one after the other, and at each clock edge. This is how flip-flops change states in theory: In an FSM model, a transition is not continuous, it occurs after the input to the FSM has been evaluated at clock time. In Simulink, such discrete transitions are implemented by Unit Delay blocks and discrete transitions inside Stateflow.

10.2.2 Clock Signal (clk_in)

The clock signal clk_in is the idealized clock used in the design of the sequential circuit. To build a D flip-flop model, we write the update:

- Q(next) = D(current) at an occurrence of a clock edge

In the simulation we do not specify the "Q(next)" for the PLC state, because the current state is retained automatically by Stateflow. Because the machine subsystems use clocked logic and a memory element, they require clk_in to update the internal registers stepwise.

So, in the simulation, clk_in represents the ideal clock that will be used to drive the FSM and machine.

10.2.3 Use of Unit Delay (1/z)

In theory, memory is a flip-flop that holds the previous value until the next clock cycle. In Simulink's unit delay blocks, the memory of "previous values" is represented as the Unit Delay (1/z) block. This is also known as "state memory". In Simulink, the Unit Delay block stores the last input value to it.

For example, in a machine subsystem, a Unit Delay is used to store the previous value of the count. This means:

- This count for time k depends upon the count for time k-1.

- This is effectively a counter in sequential logic.

Therefore, in mapping terms:

- Theoretical memory element / flip-flop storage as represented in Simulink by Unit Delay block (1/z)

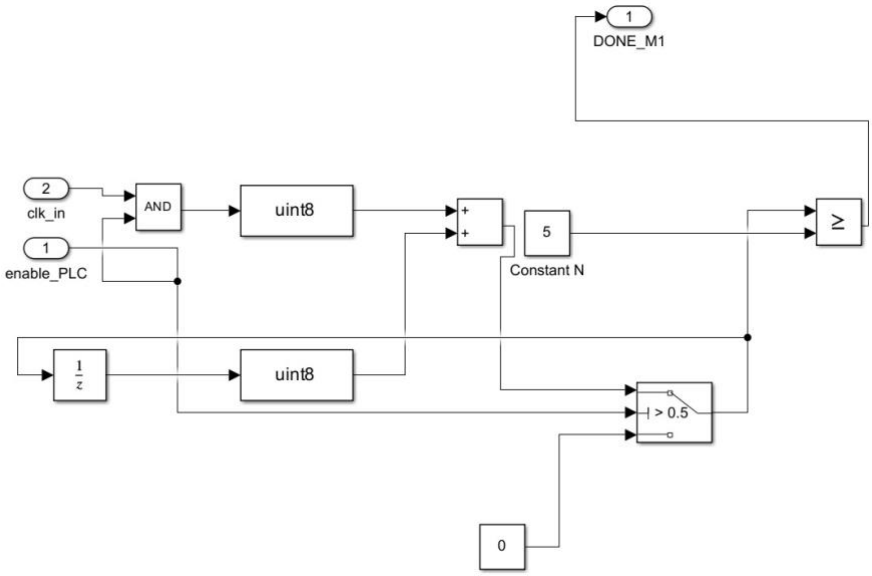

Figure 10.2 DONE_M1 machine control logic

10.2.4 Use of Flip-Flops in Machine Subsystems (DFF Blocks)

Several D flip-flop blocks (DFF_Q0, DFF_Q1, DFF_Q2) were used throughout the machine subsystems; theoretically, flip-flops could also be used for both state bits and counters. These DFFs are understood to be 1-bit registers in simulation and updated at each clock event.

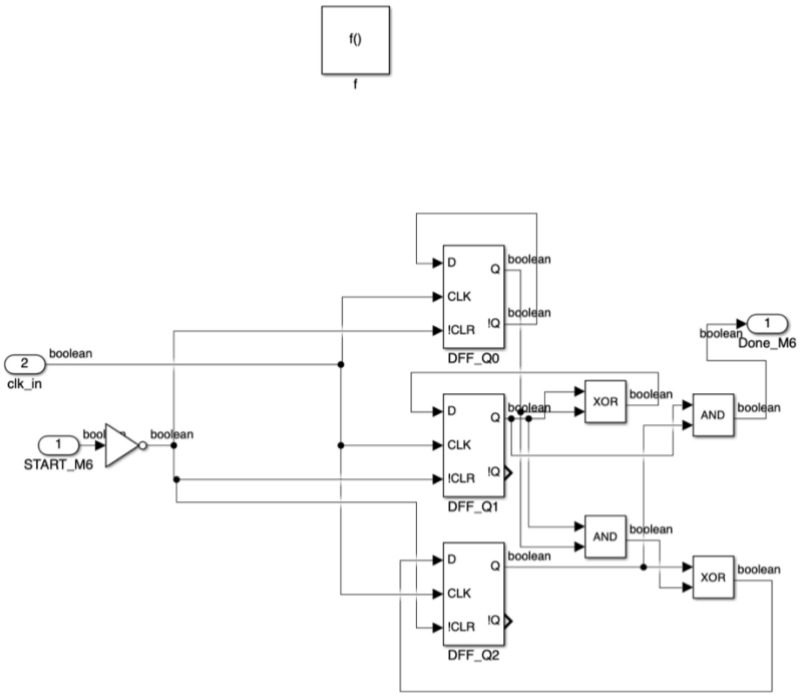

Figure 10.3 M6 function and state logic implenentation

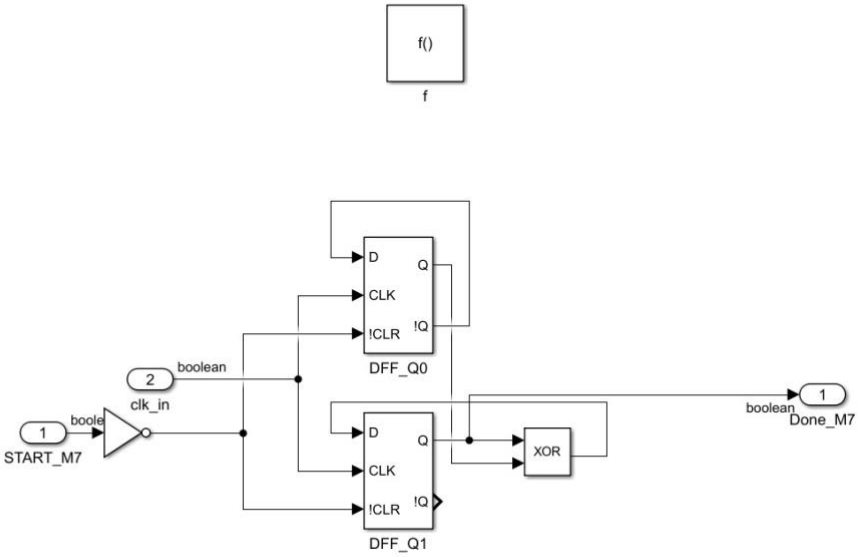

Figure 10.4 M7 function and state logic implenentation

So the mapping is:

- Theoretical D flip-flops (state bits or counter bits) → Simulink DFF blocks within each machine

Logic gates like AND, OR, XOR, and NOT were also in use. These logic gates implement combinational logic. Consequently, their next signals are a function of their current signals only. A simulation model might use Simulink Logical Operator blocks and a gate block for this.

10.2.5 Parameter N (Machine Duration)

For our machines, we can define a constant N, the number of clock cycles for a machine to be declared done. Thus, as a machine needs some processing time, we can say that in our simulation it needs N clock pulses to finish its operation.

So mapping:

- Theoretical "machine processing time" = "Constant N" set in Simulink compared to the counter.

For example, in Machine_M1, if N is set to 5, then Done_M1 will be set (asserted) after 5 clock cycles with the machine enabled.

10.3 Input Test Scenarios

To ensure correctness, we created input test scenarios that are representative of PLC operation, and verified their state transition behavior.

Scenario 1: P1/P2 Selection Signal (Product Request)

The input signal for the PLC chart is based on whether the production line should start for this particular product path. In the newest diagrams either P1 or P2 would be indicated depending on which part is passing through. This can be considered the input of the external FSM triggering leaving IDLE state.

In simulation, this input is given as a boolean constant or switch (0 or 1). This means:

- When input = 0, PLC should remain in IDLE (no operation).
- When input = 1, once the PLC reaches M1, it should sequentially proceed to M5, M3, M6, M7 (according to our FSM).

So mapping:

- Theoretical FSM input (start request / product selection) → Simulink boolean input (P1 or P2) connected to the Stateflow input port.

Scenario 2: Clock Behavior and Machine Enable

In this case, for this machine enable signal START_Mx = 1 the machine is supposed to count the clock pulses until the signal DONE_Mx = 1.

So mapping:

- Theoretical: "machine runs when active state output is 1". Simulink: "enable_PLC" input signal is true.
- Theoretically, "done becomes 1 after process time" --> In Simulink "DONE output becomes true after N clock cycles"

Scenario 3: Full Cycle Verification (Complete FSM Loop)

We have done everything:

- IDLE→M1→M5→M3→M6→M7→IDLE

In our FSM, after the final machine terminates the system returns to the IDLE state. We could check in the simulation that the chart left M7 upon the Done_M7 signal and returned back to IDLE. This indicates that the overall controller is not stuck.

10.4 Simulation Results

Stateflow animation, scopes, and output indicators were used to visualize the simulation results.

10.4.1 Stateflow Active State Observation

When running the simulation, Stateflow highlights the state currently being executed.

- If start input is 0, the active state stays in IDLE.
- If start input is 1, we transition to M1.
- When Done_M1 becomes true, the chart transitions to M5.
- It then returns from M3 through M6 and M7 back to IDLE.

This result matches the sequence of overall states outlined in the state transition design.

So mapping:

- Theoretical state sequence → Stateflow active state highlight sequence

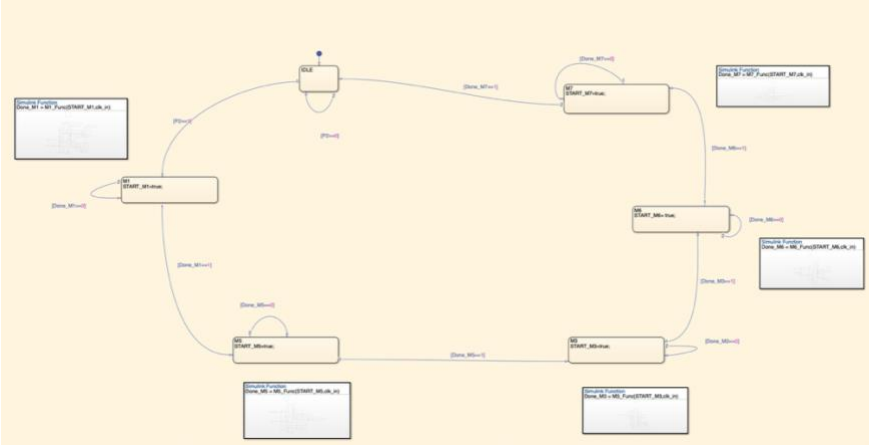

Figure 10.5 Stateflow FSM for sequential control

10.4.2 START Signals Behavior

In each state, the corresponding START signal is asserted. For example:

- In state M1 only START_M1 is true and the other START signals are false.
- START_M5 is true, and all others are false, in state M5.

This will agree with the theoretical FSM output equations as only one machine enable can be high at a time.

So mapping:

- Theoretical output vector (enabled machine) → Simulink START_Mx output vector in Stateflow

10.4.3 DONE Signals Behavior

Each machine produces a DONE signal when it has fully completed its operation. In simulation:

- After receiving enable_PLC = true, Machine_M1 counts and sets DONE_M1 = true when N clock cycles have elapsed since enable_PLC.
- Stateflow then satisfies the transition criterion [Done_M1 == true].

So mapping:

For "Done condition = 1 triggers next state" in Simulink, DONE_Mx output triggers Stateflow transition. Mealy-type FSM, in which the delay for a transition can be negligible.

10.4.4 Machine Internal Timing Behavior

Within each machine, the counter-like behavior increments on clock pulses only when the enable signal is asserted, or resets to initial state or maintains its current state depending on whether the machine was defined with switch and feedback.

This matches the theoretical sequential logic: the machine only moves when it is enabled.

10.5 Verification of Correct Operation

A critical step in the project is verifying that the system works as intended, or that the simulated system behaves exactly as was intended in the theoretical design. As part of this process we check that the simulation is running without error and that all logic, timings and order of operation are as expected. Verification was done by analyzing the transitions between states, control signals, machine completion and system flow.

The correctness of the FSM state transitions was verified first. The Stateflow animation feature of the integrated Stateflow chart visualization showed the active state throughout simulation. We see that the FSM starts in the IDLE state and remains in this state until a start condition applies if we step through this animation frame by frame. This is consistent with our theoretical view of the FSM, which states that the default state is in IDLE and that the FSM cannot function until valid input is present. This also confirms that both the initial state selection and default state selection are correct.

Upon assertion of a start input (P1 or P2 depending on the system configuration), the system will transition from the IDLE state to the first machine state M1. We checked the state transition condition label in the Stateflow file and validated that the transition from IDLE state to state M1 only happens when the condition gets satisfied. This action uses the first state-transition rule of an FSM: if start condition = 1, transition from state IDLE to state M1. Thus, implicitly, the entry condition into the production sequence is verified.

The only control signal checked after entering the M1 state was START_M1 which became true while all other START signals were false. This shows that while the PLC is in the M1 state, only one machine can be energized at any one time. This is a fundamental assumption of the sequential control idea: Only one output corresponding to the active state should be

logically asserted in the theoretical FSM output logic. The simulation shows that this rule is satisfied and the output is decoded correctly.

Next, machine execution and completion was checked, with each machine subsystem controlled independently. Machine_M1 starts to execute as soon as its start condition (START_M1) is true. They simulate clock-driven logic, memory elements (such as flip-flops), and unit delay blocks. Thus, we check whether Machine_M1 drives the signal DONE_M1 true after the expected processing time, i.e., after N clock cycles, and not before. In other words, we describe that after the processing time elapsed, the DONE_M1 becomes true. With this, the assumption of the machine requiring time to finish its task was indeed confirmed to be represented within simulation.

The above tests were also carried out on other machines (the M5, M3, M6 and the M7 machines) with similar results:

- The machine will operate when its corresponding START signal is high (true).
- The internal counter or internal memory increments with each clock pulse.
- The DONE signal is then asserted after the appropriate number of cycles.

This behavior both supports the timing model discussed above and validates that machine subsystems are correct.

An additional verification step consisted in verifying the conditions of transitions determined by DONE signals. In the theoretical FSM design, transitions between machine states were determined using Done_Mx flags. In the corresponding simulation, these flags are treated as boolean inputs to the Stateflow chart. We have verified that a state transition such as M1 to M5 occurs only if Done_M1 is true. If Done_M1 is false, the FSM remains in state M1. Thus, the FSM only follows the transitions the designer intended.

Also, we could fully operate the system in sequence because the simulation ran for that duration.

- IDLE to M1 to M5 to M3 to M6 to M7 to IDLE

If we follow the states as they transition and pay attention to the value from the START signal and the DONE signal, we can check that the FSM can return toward the IDLE state after the last machine finishes its task. This is an important property because it guarantees the FSM does not get stuck in a state and the system can operate for several production cycles.

Also checked were signal integrity and data types, and boolean signals with values 0 and 1 were found to be all of the START and DONE signals. This matches the theoretical design that states control (and status) signals are boolean in nature. These types must be used correctly,

because, e.g. if double is used in place of boolean, the behavior changes considerably. If the final simulation contains no type errors, the signal definitions are deemed correct.

It was confirmed that the clocking was synchronous. All of the logic subsystems are clocked by the same clock (clk_in), and all were synchronous with the clock. The clock edges results showed that the system was synchronous digital and implemented the theoretical design model of the clocked flip-flops of sequential circuits.

There are also signal views, Stateflow highlighting, and outputs to make it easy to see the cause and effect of the implementation. For example, with a START signal a machine starts, and with a DONE signal the machine transitions into a different state. We verified that the simulated model implements the theoretical FSM design and meets the project's functional requirements.

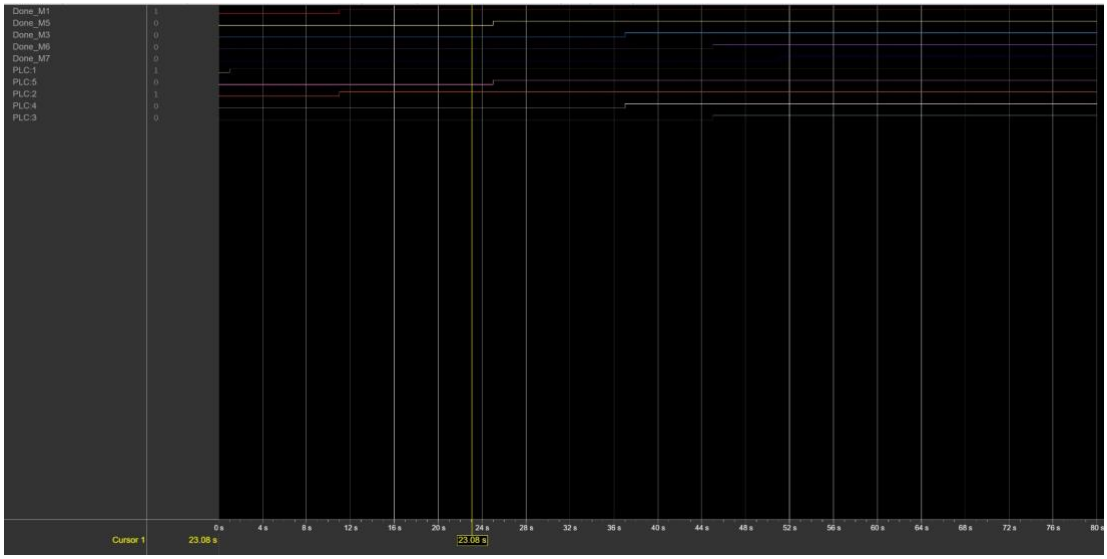

Figure 10.6 Logic Analyzer – FSM State Transitions

11 DISCUSSION

11.1 Design Observations

Many observations have been made while designing and simulating the production control system. The most important of which being that the Stateflow environment allows for a clearer purpose for the Finite State Machine (FSM) than a simple set of logic gates. While the theoretical design describes the FSM behavior on the basis of state tables, state encoding, and transition conditions, these concepts are more intuitively represented as states and transitions in the Stateflow chart. Because each state corresponds to a specific machine operation, the control logic is better organized and easier to analyze.

Another important aspect is the clock-driven behavior of the system where a global clock signal is used to synchronize all machines and control logic. This matches the assumption of a synchronous sequential system composed of flip-flops in the theoretical model. Since memory elements such as D flip-flops and Unit Delay blocks store their previous values and update them only after a clock event, this avoids a race condition and ensures that machine completion signals do not occur before this processing time has elapsed.

The modular structure of the design was found to be helpful as each machine is implemented as its own subsystem. This made it easy to test and debug the final model. The design reflects good engineering practice and is similar to a real PLC-based system where machines and controllers are separate modules.

11.2 Advantages of the Proposed FSM

A FSM-based control system has many advantages. The most important is its deterministic behavior. A FSM has a defined sequence of states and transitions between states. This means that a system based on an FSM has predictable and repeatable behavior. This is especially powerful because poor sequencing will cause errors or unsafe conditions in production.

Another benefit is the separation of control logic from machine behavior: the decision and sequencing logic is implemented in the Stateflow chart, while the machine subsystems deal with processing time and completion. As a result, it is simpler to understand and modify, and

the boolean START and DONE signals offer a simple and reliable method for the PLC to communicate with the machines it is controlling.

The design of the FSM guarantees that only a single machine is active at any specific time, avoiding resource contention. Likewise, theoretical FSM output logic guarantees that only a single output line corresponding to the current state is active. Thus the system operates in a more controlled and safer manner.

11.3 Possible Improvements

Although the proposed design satisfies all specification requirements, the design could be further improved by adding fault detection and timeout features to make the system more strong, since in a production environment a machine may not complete its operation within the expected amount of time. By adding timeout and error states to the FSM, such problems could be detected and appropriate actions could be taken.

An option to improve on this is to make the system more flexible by using parameters instead of fixed properties (i.e., using parameters for the number of clock cycles needed by a machine to execute a command) so that the control structure can be reused for different production processes with little change.

Finally the system could be extended to include more complex production processes such as parallel operation of machines or simultaneous production of multiple products. This would require a more complex FSM definition, but would make the system much more suitable for industry use.

12 CONCLUSION

12.1 Summary of the Design

So we did it. We created a PLC-controlled production sequence based on a finite state machine where the states and their transitions were implemented in the order of the process. Then we implemented the controller in Stateflow and implemented machine behaviors using flip-flop logic in Simulink.

The PLC controller issues START commands to initiate each machine. Each machine uses clock-driven memory (flip-flops / unit delay) to simulate a processing delay by a set number of cycles, before issuing DONE commands. The PLC uses the presence of DONE signals to determine which state to enter.

Most importantly, the design simulated well and represents the theoretical design:

- FSM states → Stateflow states
- Transition conditions → guard conditions on arrows
- binary signals → boolean signals
- Memory (flip-flops) → DFF and Unit Delay
- Processing time → N cycles counter/comparator logic

This creates a self-contained, internally consistent model.

12.2 Final Remarks

The main purpose of the project was to investigate the interface between theoretical digital logic and practical simulation design. States and transitions of an FSM require careful definitions to achieve deterministic behavior. As such, in this simulated case, we confirm that these theoretical decisions are not just mathematical abstractions, but also physically relevant.

A very important lesson is that memory and timing are an integral part of any sequential system. Without some kind of memory elements like flip-flops or unit delay, the system cannot "remember" its previous state. Real digital circuitry uses flip-flops to store the machine state. These were represented by DFF blocks and Unit Delay blocks in Simulink. Thus digital machines could be constructed and executed step by step under the control of clock pulses.

PLC control means not only enabling the signals, but also feedback. In this closed-loop system, the PLC enables a machine, and waits for a DONE feedback. This feedback is needed to guarantee the correct operation of the sequence. DONE signals are the conditions in

theoretical FSM tables. In the simulation, they are real signals emitted by the machine subsystems, making this design approach more realistic and more easily verifiable.

Also, this project showed us that the signal data types matter. Digital systems work 0/1. Therefore, boolean types must be used, because if the signals become double, it will create errors or unexpected behavior in the model. A correct engineering simulation must account not only for logic connections, but also data types, sample times and clocking.

Finally, our analysis opens the door to future work, as our FSM structure could be extended with additional states, new machines, error handling, and safety checks. This system is fairly modular, which means improving it is easy. A key engineering skill is designing something that works now, but which is easy to improve later. The project becomes more than just a simulation task, as it is a whole practical learning experience about digital control, PLC-like systems, and FSM-based automation system design.

REFERENCES

[1] Mano, M. Morris, and Michael D. Ciletti. *Digital Design*. 4th ed., Prentice Hall International, 2007.

[2] Vahid, Frank. *Digital Design with RTL Design, VHDL & Verilog*. 2nd ed., Wiley, 2011.

[3] Arsan, Taner, and Rifat Çölkesen. *Lojik Devre Tasarımı*. 4. baskı, Papatya Yayıncılık, 2007.